

Lightweight Trap-Based Free-Rider Detection and Mitigation in Federated Learning

Aashnan Rahman

Department of Computer Science and Engineering

Islamic University of Technology

MSc Thesis

Abstract—Free riders in federated learning receive a shared model while avoiding local optimization. This paper presents a lightweight server-side challenge protocol combining undisclosed model perturbations, robust update statistics, compact history reconstruction, coherent cohort evidence, and repeated confirmation. The detector operates on attributable returned updates without client-reported training metrics, auxiliary learned detectors, public canary data, or specialized hardware. Thirty-two-dimensional sketches test whether returned updates are explainable from models previously sent to each client, while reference probation and reversible suspicion states limit decisions based on isolated anomalies. The method is evaluated in 16 configurations with 100 clients, 100 rounds, and 40% free riders, covering four attacks on MNIST and CIFAR-10 under IID and Dirichlet non-IID partitions. All 40 attackers are removed in every run, by round 10 for playback and round 30 for the other attacks. Fifteen configurations have no honest removals; CIFAR-10 non-IID historical averaging removes one honest client, yielding 97.56% precision and 98.77% F1. Mean final model accuracy is 98.85%/98.83% for IID/non-IID MNIST and 76.18%/74.71% for CIFAR-10. These results demonstrate feasibility under the evaluated threat model while identifying heterogeneous honest updates and temporary quarantine as remaining limitations.

Index Terms—federated learning, free riding, challenge-response, robust statistics, history reconstruction

I. INTRODUCTION

Federated learning (FL) enables multiple clients to train a shared model without transferring their raw data to a central server. In a typical communication round, the server distributes the current global model, participating clients improve it using their local data, and the server aggregates their returned updates, for example through Federated Averaging (FedAvg) [1]. This collaborative arrangement makes learning possible across distributed and privacy-sensitive data sources and has supported large-scale applications [2], [3].

The quality of the resulting global model nevertheless depends on clients actually performing the local work expected of them. In practice, some clients may decide not to contribute meaningful training. A client may have insufficient useful data, limited computation or energy, or may simply wish to conserve its own resources while continuing to receive the model produced by the federation. Such a participant is commonly described as a *free rider* [4], [5]. It obtains the benefit of the collaborative model while contributing little or no genuine learning in return.

Free riding differs from an active poisoning attack. A free rider does not necessarily attempt to insert a harmful objective, corrupt labels, or deliberately steer the global model in an adversarial direction. Its harmful effect is primarily an omission of useful work. Had the client trained honestly, its local data and computation could have produced an informative update and potentially improved the aggregated model. When it does not, the global model must depend more heavily on the remaining contributors. One inactive participant may have little visible effect, but widespread free riding reduces the effective training population, wastes communication and coordination resources, slows or weakens learning, and creates an unfair distribution of computational cost.

Accordingly, this work studies a lightweight server-side method for identifying clients that repeatedly avoid meaningful local training without relying on client-reported performance or resource information.

The principal contributions are:

- **Lightweight server-side detection:** The method uses attributable model updates already available to the server. It requests no client accuracy, loss, resource report, or execution log and trains no auxiliary detector.
- **Secret trap-based auditing:** Selected clients receive undisclosed, scale-aware perturbations. Responses to the same challenge provide controlled evidence without revealing the audit schedule or requiring clients to train an additional model.
- **Compact history and repeated confirmation:** Thirty-two-dimensional sketches test whether returned updates are explainable by previously sent models. Coherent cohort evidence, trusted-reference confirmation, probation, quarantine, and rehabilitation prevent one unusual response from directly causing removal.
- **Evaluation across attacks and data settings:** Four free-rider strategies are evaluated on MNIST and CIFAR-10 under IID and Dirichlet non-IID partitions. The evaluation reports detection, removal timing, model utility, execution cost, and residual candidate states.

II. RELATED WORK

A. Free-rider attacks

Lin *et al.* formalized free riding in FL and introduced both fabricated-update attacks and STD-DAGMM detection [4]. Fraboni *et al.* showed that free riders can imitate successive

global updates and disguise them with calibrated noise [5]. Zhu *et al.* subsequently constructed a stronger attack that matches geometric properties used by feature-based defenses [6]. Collectively, these works show that replay and arbitrary noise are insufficient evaluation targets: a detector based on a fixed, known update statistic can be imitated by a history-aware attacker.

B. Statistical and Learning-Based Detection

Statistical detectors use update norms, dispersion, cosine similarity, or temporal consistency because these signals are available at the server and relatively inexpensive. Their central weakness is that non-IID honest updates may also be outliers, while adaptive attackers can reproduce the statistics being monitored. **FRAD** improves on a single statistic by combining a DAGMM-style anomaly detector with contribution and reputation models [7]. **PASS** combines parameter auditing with contribution evaluation and considers both resource-poor and strategically selfish free riders [8]. Both enrich the evidence available to the server, but introduce auxiliary models or auditing assumptions, additional calibration, and associated computational work. Their learned or engineered references may also require adaptation when the model architecture, data heterogeneity, or attacker behavior changes.

C. Challenge-Based and Verifiable Computation

FRIDA seeks evidence of genuine training through privacy-attack techniques [9]. Its membership-inference variants use a shared canary batch, while its property-inference variants test whether inferred client-data properties remain plausible across rounds. This targets the cause of free riding more directly than update geometry, but adds client or server computation, requires individual-update access, and introduces an additional canary objective whose robustness requires evaluation. Trusted hardware, remote attestation, and cryptographic proof can provide stronger execution guarantees, but require specialized support or substantial proof and communication overhead. They also complicate integration with secure aggregation [10]. The remaining gap is a lightweight server-side method that creates unpredictable evidence without auxiliary detector training, trusted client reports, public canary data, or specialized hardware.

The comparisons above concern design assumptions. This evaluation does not establish a quantitative accuracy or runtime advantage over these defenses, because no matched baseline experiment is included in the reported matrix.

III. THREAT MODEL AND ATTACKS

There are N attributable clients and a coordinating server. In round t , client i receives $w_{t,i}^{\text{send}}$ and returns $w_{t,i}^{\text{ret}}$. The server derives

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \quad r_{t,i} = \|\Delta_{t,i}\|_2. \quad (1)$$

For an audited client, $w_{t,i}^{\text{send}}$ is a perturbed trap rather than the aggregating global model. Clients receive no trap indicator, and trap-derived updates are excluded from aggregation.

Attackers know only models previously sent to them. They do not know the audit schedule, group membership, reference-set membership, thresholds, or ground-truth labels. Four increasingly informed behaviors are evaluated.

FR1: Playback

$$\hat{w}_{t,i} = w_{t-1,i}^{\text{send}}. \quad (2)$$

The client stores the last model it received and submits that model in the next round without training. During its first round, when no previous model exists, it returns the current received model. This is the least expensive attack, but it responds to a new model using stale information and may also create an exact or near-exact send-back.

FR2: Bounded Random Update

$$\hat{w}_{t,i} = w_{t,i}^{\text{send}} + \epsilon, \quad \epsilon_j \sim \mathcal{U}(-10^{-3}, 10^{-3}). \quad (3)$$

FR2 avoids an exact send-back by adding an independently sampled bounded value to every floating-point coordinate. The response is nonzero and superficially update-like, but it contains no information from local data and has no memory of earlier rounds. It represents a client attempting to bypass simple equality or zero-norm checks at negligible computational cost.

FR3: Historical Averaging

$$\hat{w}_{t,i} = \frac{1}{|\mathcal{H}_{t,i}|} \sum_{w \in \mathcal{H}_{t,i}} w, \quad |\mathcal{H}_{t,i}| \leq 5. \quad (4)$$

The client maintains a rolling history $\mathcal{H}_{t,i}$ of up to five models received from the server, including the current one, and returns their element-wise mean. Averaging produces a smooth, history-dependent response without local optimization. Because the client is not told which models are traps, every trap enters the same history and can affect several later responses.

FR4: Trajectory Imitation

$$d_s = w_{s,i}^{\text{send}} - w_{s-1,i}^{\text{send}}, \quad \hat{w}_{t,i} = w_{t,i}^{\text{send}} + \bar{d} + \eta, \quad (5)$$

where $\bar{d}$ is the mean of the latest five received-model differences and Gaussian η is normalized to 10% of $\|\bar{d}\|_2$. At cold start, the attacker uses a small Gaussian update. FR4 therefore attempts to imitate the direction and scale of recent global evolution, making it more plausible than memoryless noise. Nevertheless, it uses every received model indiscriminately, so an undisclosed trap also contaminates its trajectory estimate.

Collusion, partial honest training, intermittent participation, and defense-aware adaptation are outside the evaluated scope.

IV. PROPOSED METHODOLOGY

Detection primitives. For floating tensor k , the server constructs a scale-aware challenge

$$\tau_k = w_{t,k} + 0.01 \max(\sigma(w_{t,k}), 10^{-4}) \xi_k, \quad \xi_k \sim \mathcal{N}(0, I). \quad (6)$$

Returned updates are measured relative to the model actually sent to each client. Their norm and normalized layer profile are

$$r_i = \|\Delta_i\|_2, \quad p_{i,k} = \frac{\|\Delta_{i,k}\|_2}{\max(\sum_j \|\Delta_{i,j}\|_2, \epsilon)}. \quad (7)$$

Algorithm 1 Phase I: warm-up and passive evidence

Require: global state w_t , active clients V_t

- 1: **for all** $i \in V_t$ **do**
 - 2: send w_t and receive $w_{t,i}^{\text{ret}}$
 - 3: compute $\Delta_{t,i}$ and the sent-model fingerprint
 - 4: **if** $\|\Delta_{t,i}\|_2 < 10^{-6}$ or an exact prior-send replay occurs **then**
 - 5: add one permanent send-back strike, at most once this round
 - 6: **end if**
 - 7: **end for**
 - 8: **if** $t = 10$ **then**
 - 9: remove every client with at least five send-back strikes
 - 10: **end if**
 - 11: aggregate the remaining eligible ordinary responses
-

Responses to the same challenge form a comparison group. Update norms are tested on both tails; distance from the componentwise median layer profile is tested on the upper tail. Robust scales use the normal-consistency-corrected MAD, a numerical floor, and historical scale stabilization. A nominal false-alarm budget of 0.01 sets cycle cutoffs, with prior reference-cycle calibration for the static tests. This calibration parameter is not a theorem guaranteeing a 1% false-removal rate under dependent, heterogeneous observations.

A. Phase I: Warm-Up

The first ten rounds provide ordinary training and passive send-back checks. Near-zero updates and exact replay of recorded sent models contribute to the passive evidence stream. Removal under this rule requires at least five accumulated send-back events and cannot occur before the ten-round warm-up boundary. The passive path remains available after warm-up. Attack labels are needed to simulate attacks and score outcomes, not to choose detector decisions. The procedure is summarized in Algorithm 1.

B. Phase II: Double Coverage

After warm-up, the server schedules distinct secret challenges and accumulates paired observations for each client. The initial reference uses the active population; subsequent comparisons use eligible reference clients when available. A trusted comparison requires at least ten reference clients. Here “trusted” denotes an operational reference state inferred by the protocol, not known honest ground truth.

The server also stores compact sketches. Let S denote the implementation’s deterministic signed linear map of width 32, $s_{t,i} = S(w_{t,i}^{\text{send}})$, and $u_{t,i} = S(\Delta_{t,i})$. Candidate reconstructions use only the client’s sent-model history:

$$q_{t,i}^{\text{mean}}(m) = \frac{1}{m} \sum_{j=0}^{m-1} s_{t-j,i} - s_{t,i}, \quad (8)$$

$$q_{t,i}^{\text{traj}} = \text{mean}_j(s_{j,i} - s_{j-1,i}). \quad (9)$$

Algorithm 2 Phase II: double coverage and evidence construction

Require: active clients V , distinct traps τ_A, τ_B

- 1: partition V into ten hidden, balanced groups
 - 2: schedule two groups per round over two randomized passes
 - 3: **for all** $\tau \in \{\tau_A, \tau_B\}$ **do**
 - 4: **for all** groups assigned to τ **do**
 - 5: send τ ; retain norm, layer profile, and compact sketches
 - 6: exclude challenged responses from aggregation
 - 7: **end for**
 - 8: freeze a same-trap reference and robust calibration statistics
 - 9: **end for**
 - 10: **for all** $i \in V$ **do**
 - 11: test norm, layer profile, mean history, and trajectory history
 - 12: retain coherent signatures across the paired challenges
 - 13: **end for**
 - 14: discard completed-cycle response evidence after state updates
-

The history window is five; mean hypotheses consider available lengths from two through five. Similarity is scored by

$$h(u, q) = -\log \max \left(\frac{\|u - q\|_2}{\max(\|u\|_2 + \|q\|_2, \epsilon)}, \epsilon \right). \quad (10)$$

The mean-history score takes the maximum over eligible windows. High scores indicate explainability from server-sent history rather than establish proof of misconduct. History scores use one-sided robust calibration based on the median and lower-side deviations to limit inflation by a high-scoring minority. Sketches reduce retained history size but still require reading model coordinates; they are not a constant-time detector or cryptographic commitment. Algorithm 2 specifies the double-coverage cycle.

C. Phase III: Surveillance and Candidate Confirmation

A signature is coherent when it persists across paired challenges. Strong evidence may arise from coherent joint norm/profile anomalies, a coherent static-signature cohort, or a coherent history-signature cohort. The cohort threshold is calculated from the active population and a binomial upper-tail criterion with the configured false-alarm budget. It is not a requested attacker count or a fixed target of 40 removals. When a sufficient mean-replay cohort is present, the broader trajectory signature is suppressed to reduce ambiguous attribution of ordinary training movement.

Clients occupy reference (R), candidate (C), or suspect (S) roles. An isolated weak anomaly can produce a candidate without a score penalty. A previously eligible reference client first enters C when anomalous and must be independently checked again. New strong evidence maintains or restores double probing; sustained clean cycles allow single probing and then dormant intervals. Dormancy retains passive send-back

Algorithm 3 Phase III: surveillance and confirmation

```

1: if a double cycle produces new coherent strong evidence then
2:   retain double probing and update  $R/C/S$  roles
3: else if two consecutive double cycles are clean then
4:   enter single-probe surveillance
5: end if
6: if a single cycle produces an anomaly then
7:   return to double probing
8: else if two consecutive single cycles are clean then
9:   enter a 20-round dormant interval
10: end if
11: retain passive send-back checks during dormancy
12: run one single audit cycle when dormancy expires

```

checks and periodically reactivates challenges. The configured relaxation requires two clean cycles at each step and uses a 20-round dormant interval. The transition logic appears in Algorithm 3.

D. Phase IV: Suspicion, Penalties, and Rehabilitation

Strong cycles decrement the score by two; weak candidate evidence contributes zero. A clean cycle restores 0.5 toward a maximum of zero. Statistical removal requires a score at or below -4 , at least two strong cycles, and at least one strong trusted-reference confirmation after the initial cycle. The separate passive send-back criterion can also remove a client.

Two clean cycles demote S to C or clear C to R , with reference probation before renewed eligibility. Candidate and suspect clients, challenged clients, and clients removed in the current round are excluded as prescribed by the aggregation path. Therefore a client may remain active but contribute less often: active-client count and aggregate-contributor count must not be conflated. The method retains client counters, reference/calibration histories, and compact model-history sketches in addition to ordinary training state. Algorithm 4 summarizes scoring, removal, and rehabilitation.

Lightweight implementation. For a model with P floating parameters and L tensors, the server already reads each attributable response to form the model difference. During that pass it computes a scalar norm, an L -dimensional normalized profile, and a fixed 32-value signed sketch. The bounded history window of five therefore stores compact sketches rather than full historical models or gradients for each client. Median/MAD calibration, reconstruction comparisons, cohort counting, and state transitions operate on these small summaries. The server trains no auxiliary detector, and the client performs ordinary local optimization on the model it receives rather than training a second model. The principal operational costs are challenge generation, probe training, server-side parameter scans, and temporary exclusion of challenged or quarantined responses.

Algorithm 4 Phase IV: suspicion, removal, and rehabilitation

```

1: for all clients evaluated at a completed cycle do
2:   if coherent strong evidence is present then
3:     decrement score by two; reset the clean-cycle streak
4:   else if only weak evidence is present then
5:     place or retain the client in  $C$  without a score cost
6:   else
7:     restore 0.5 toward a maximum score of zero
8:     increment the role-specific clean-cycle streak
9:   end if
10:  if score  $\leq -4$ , two strong cycles, and trusted confirmation then
11:    remove the client
12:  else if the role-specific two-clean-cycle rule is met then
13:    demote  $S \rightarrow C$  or clear  $C \rightarrow R$ 
14:    apply one clean reference-probation cycle
15:  end if
16: end for
17: independently remove clients reaching five send-back strikes
18: aggregate only eligible ordinary responses

```

V. EXPERIMENTAL DESIGN

MNIST [11] and CIFAR-10 [12] each use 100 initially participating clients, 100 rounds, three local epochs, batch size 32, and SGD with learning rate 0.01 and momentum 0.9. The MNIST model has two convolutional blocks (16 and 32 channels) and a 128-unit hidden classifier; CIFAR-10 uses three convolutional blocks (32, 64, and 128 channels) with batch normalization and a 256-unit hidden classifier. IID partitions and Dirichlet non-IID partitions with $\alpha = 0.5$ are evaluated. Each run contains one attack type and exactly 40 free riders selected at seed 42. All 16 configurations are reported, not best-of-seed selections.

The experiments execute with CUDA on an NVIDIA GeForce GTX 1050 Ti. Per-round timings include client training and protocol work rather than isolated detector cost. Source configurations and completed-run markers are retained with the results.

For final removal, TP counts removed free riders, FP removed honest clients, and FN remaining free riders. Precision is $TP/(TP + FP)$, recall is $TP/(TP + FN)$, and $F_1 = 2PR/(P + R)$. False-positive removal rate is $FP/60$. Model accuracy is test classification accuracy, distinct from detection accuracy. Counts are cross-checked against unique client IDs in removal logs; progress curves are derived from event rounds rather than inferred from final summaries.

VI. RESULTS

A. Detection Performance

Table I reports all evaluated configurations. Every run removes all 40 attackers. Fifteen runs remove no honest client; CIFAR-10 non-IID FR3 removes one honest client, producing 40 true positives, one false positive, and zero false negatives. Its

precision is 97.56%, F1 is 98.77%, and honest-client removal rate is 1.67%. Summing across runs gives 640 removed attacker instances, not 640 distinct real-world participants.

B. Detection Progress

All 40 FR1 clients are removed at round 10 in each dataset/partition setting. All 40 FR2, FR3, and FR4 clients are removed at round 30. These coincident removal rounds reflect batch decision boundaries in this configuration rather than continuous per-response removal. Figure 1 reconstructs these curves from the event logs. Timing alone does not identify which signature caused each decision; causal attribution would require ablations.

C. Comparative Result Analysis

Effect of dataset: MNIST final accuracy remains tightly grouped around 98.8%, and no MNIST run removes an honest client. CIFAR-10 is more demanding: mean final accuracy is 76.18% under IID allocation and 74.71% under non-IID allocation. The only permanent false removal also occurs on CIFAR-10. This suggests that the larger variation in CIFAR-10 update geometry is more challenging for robust comparison than the MNIST setting, although a matched no-attack baseline is needed to separate protocol cost from ordinary dataset difficulty.

Effect of data partitioning: MNIST final accuracy is similar across distributions: 98.85% averaged over the four IID attacks and 98.83% over non-IID attacks. The attack-matched non-IID changes are -0.05 , -0.02 , -0.01 , and -0.01 percentage points for FR1–FR4. CIFAR-10 mean final accuracy falls by 1.47 points under non-IID allocation. Non-IID minus IID differences for FR1–FR4 are $+0.40$, -1.97 , -2.08 , and -2.24 points. The sole false removal is honest client 32 in CIFAR-10 non-IID FR3 at round 30. Its logged basis is two strong cycles with trusted confirmation, score -4 , and zero send-back events. Reference confirmation therefore does not eliminate every heterogeneous honest outlier.

Effect of attack type: FR1 is removed at the warm-up boundary because playback repeatedly matches a model previously sent by the server. FR2, FR3, and FR4 all complete at round 30 in the present matrix, after two post-warm-up evidence cycles. Identical removal rounds do not imply identical internal evidence: bounded random fabrication, historical averaging, and trajectory imitation are tested by different observable signatures within the same attack-agnostic detector. Trace-level ablation is required before attributing a removal to the static statistics, mean-history reconstruction, trajectory reconstruction, or cohort rule.

The controlled configuration makes the 16 experiments directly comparable, but the evaluation does not provide independent repetitions or confidence intervals. The results therefore establish performance in the stated setting rather than universal reliability.

D. Utility and Execution Cost

Figure 2 shows the learning trajectories. The mean final test accuracies above measure model utility, but no no-attack

baseline is available to isolate the loss caused by free riding or by quarantine. Summed round time is 6.37/6.53 hours for the four MNIST IID/non-IID runs and 12.77/10.82 hours for CIFAR-10. These totals exclude setup overhead and are not detector-only costs; they must not be interpreted as a CUDA/CPU comparison.

Table II gives the corresponding compact execution profile. Seconds per round are averaged across the four attacks in each setting. Peak resident memory is the largest process value observed among those four runs, so it is deliberately conservative rather than an average. All rows use the same CUDA device.

Permanent removal is not the only operational cost. At termination, honest clients can remain in candidate or suspect states and be excluded from some aggregation rounds. Table III exposes these residual states. Remaining counts do not estimate an unseen attacker population, since all simulated attackers have already been removed.

VII. DISCUSSION AND LIMITATIONS

The results show complete final removal recall in the evaluated matrix and only one permanent false removal. They do not demonstrate that all honest participation is preserved: quarantine and probe exclusion can reduce useful contributions even with perfect removal precision. The 32-dimensional sketch limits history storage but does not by itself prove low end-to-end overhead, and a reconstruction match is statistical evidence rather than verification of training.

The nominal false-alarm and binomial cohort calculations are operational heuristics under shared models and correlated observations. Their independence and calibration assumptions need not hold under heterogeneous data. Ground-truth attacker identities are used for simulation and evaluation; the detector interface does not receive them. This separation does not substitute for a comprehensive implementation audit. Repeated seeds, ablations of history evidence and cohort gating, and matched comparisons with baseline defenses are required before broad comparative claims.

VIII. ASSUMPTIONS AND FUTURE WORK

The study assumes an honest server, attributable individual updates, full participation of active clients, fixed attack identities, one attack family per run, and honest execution of prescribed local optimization by nonattackers. The simulated attackers do not adapt to inferred audit state. Unmodified secure aggregation hides the individual responses this protocol needs, so deployment with secure aggregation requires additional protocol design.

Future work should evaluate partial training, intermittent participation, mixed and colluding attacks, defense-aware evasion, additional seeds, attacker proportions, heterogeneity levels, and matched no-attack and detector baselines. The first diagnostic priority is the non-IID FR3 false removal and residual honest quarantine. A controlled device benchmark must hold attacker fraction, attack, model, seed, and execution conditions fixed.

TABLE I
DETECTION AND MODEL PERFORMANCE WITH 40% FREE RIDERS. ACCURACIES AND DETECTION METRICS ARE PERCENTAGES. LAST FR IS THE ROUND BY WHICH ALL FREE RIDERS WERE REMOVED.

Dataset	Partition	Attack	Final acc.	Best acc.	Precision	Recall	F1	Honest removed	Last FR
MNIST	IID	FR1	98.87	98.91	100.00	100.00	100.00	0	10
		FR2	98.87	98.87	100.00	100.00	100.00	0	30
		FR3	98.82	98.87	100.00	100.00	100.00	0	30
		FR4	98.83	98.84	100.00	100.00	100.00	0	30
	Non-IID	FR1	98.82	98.87	100.00	100.00	100.00	0	10
		FR2	98.85	98.88	100.00	100.00	100.00	0	30
		FR3	98.81	98.82	100.00	100.00	100.00	0	30
		FR4	98.82	98.86	100.00	100.00	100.00	0	30
CIFAR-10	IID	FR1	74.62	75.53	100.00	100.00	100.00	0	10
		FR2	76.74	77.13	100.00	100.00	100.00	0	30
		FR3	76.80	76.80	100.00	100.00	100.00	0	30
		FR4	76.56	77.05	100.00	100.00	100.00	0	30
	Non-IID	FR1	75.02	75.12	100.00	100.00	100.00	0	10
		FR2	74.77	75.10	100.00	100.00	100.00	0	30
		FR3	74.72	74.77	97.56	100.00	98.77	1	30
		FR4	74.32	74.87	100.00	100.00	100.00	0	30

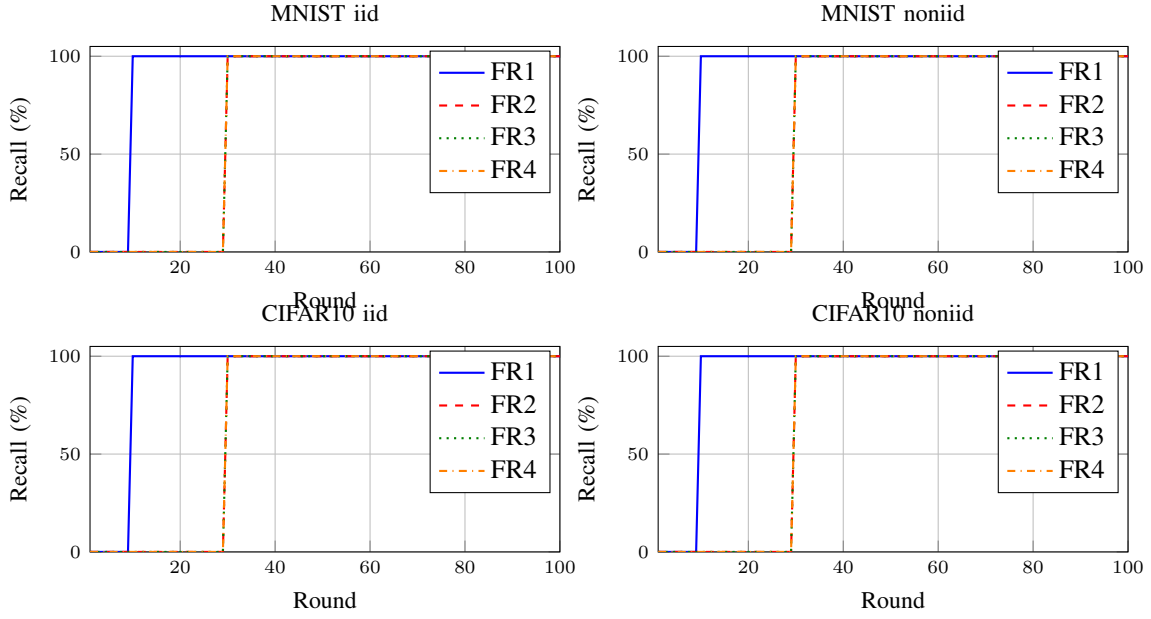


Fig. 1. Cumulative removal recall from event logs. Curves overlap where attacks share removal rounds; FR2–FR4 coincide at round 30 in every panel.

TABLE II
RUNTIME AND MEMORY USAGE FOR THE COMPLETED 40% MATRIX.

Dataset	Partition	Device	s/round	Total h	Peak RSS (GB)
MNIST	IID	CUDA	57.4	6.37	1.27
MNIST	Non-IID	CUDA	58.7	6.53	1.55
CIFAR-10	IID	CUDA	114.9	12.77	2.09
CIFAR-10	Non-IID	CUDA	97.4	10.82	3.13

TABLE III
FINAL CANDIDATE/SUSPECT COUNTS, ORDERED FR1, FR2, FR3, FR4.

Dataset	Partition	C/S by attack
MNIST	iid	5/0, 9/0, 16/0, 14/0
MNIST	noniid	10/0, 15/0, 3/0, 7/0
CIFAR10	iid	36/0, 5/0, 26/0, 15/0
CIFAR10	noniid	15/0, 19/0, 14/1, 10/0

IX. CONCLUSION

The proposed method combines secret model challenges, robust static evidence, compact history reconstruction, and

repeated reference-based confirmation. Across all 16 experiments with 40% free riders, every attacker is removed, with one

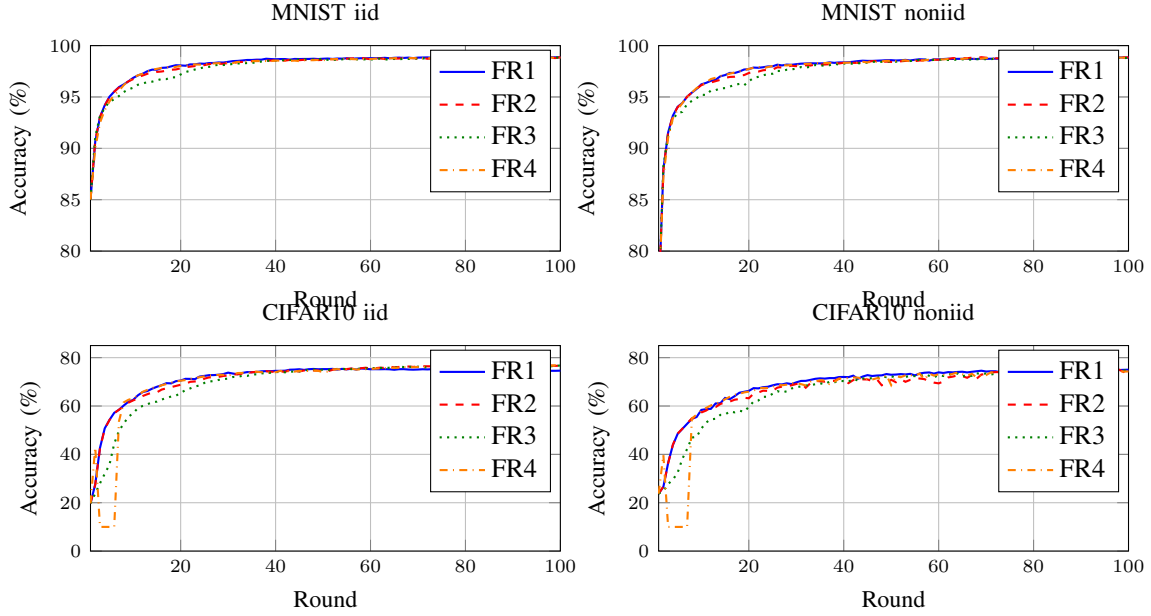


Fig. 2. Per-round test accuracy for every completed run. Panels use dataset-specific vertical ranges; compare attack curves within a panel.

honest removal in CIFAR-10 non-IID FR3. MNIST accuracy remains near 98.8%; CIFAR-10 averages 76.18% IID and 74.71% non-IID. The evidence supports feasibility in this controlled setting while retaining explicit uncertainty about generalization, adaptive attackers, transient exclusion, and computational overhead.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [2] K. Bonawitz, H. Eichner, W. Grieskamp *et al.*, "Towards federated learning at scale: System design," in *Proceedings of Machine Learning and Systems*, vol. 1, 2019, pp. 374–388.
- [3] A. Hard, K. Rao, R. Mathews *et al.*, "Federated learning for mobile keyboard prediction," *arXiv preprint arXiv:1811.03604*, 2018.
- [4] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," *arXiv preprint arXiv:1911.12560*, 2019.
- [5] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Proc. AISTATS*, 2021, pp. 1846–1854.
- [6] Z. Zhu, J. Shu, X. Zou, and X. Jia, "Advanced free-rider attacks in federated learning," in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021.
- [7] B. Wang, H. Li, X. Liu, and Y. Guo, "FRAD: Free-rider attacks detection mechanism for federated learning in AIoT," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024.
- [8] J. Wang, X. Chang, J. Mišić, V. Mišić, and Y. Wang, "PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2023.
- [9] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, "FRIDA: Free-rider detection using privacy attacks," *arXiv preprint arXiv:2410.05020*, 2024.
- [10] K. Bonawitz, V. Ivanov, B. Kreuter *et al.*, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM CCS*, 2017, pp. 1175–1191.
- [11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [12] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.